



Micro quick syntax reference

A quick syntax reference for the Micro programming language



Generic syntax

Script : Any sequence of expressions, separated by <code>;</code> , last semicolon optionnal
Comments : inline with <code>##</code> or multiline enclosed in <code>#' ... '#</code>

Operators

Arity : sets how many operands it takes
Precedence : sets who gets the parentheses
Syntax : - any combination of <code>& ^@=+-*%!\$/:.,?!<></code> <code>#alphanumeric_chars_with_no_spaces</code>
Special operators : <code>#call</code> : <code>f(...)</code> <code>#index</code> : <code>x[...]</code> <code>#list</code> : <code>[a;b;...]</code> <code>#tuple</code> : <code>(a;b;...)</code>
Primitive operators : <code>#number</code> <code>12</code> is <code>[#number '12']</code> <code>1.2</code> is <code>[#number '1.2']</code> <code>0b011</code> is <code>[#number '011' 'b']</code> <code>#string</code> <code>"s"</code> is <code>[#string 's']</code> <code>"s0 {arg1} ... {argn} sn"</code> is <code>[#string 's0 arg1 ... argn 'sn]</code> <code>#name</code> <code>identifier</code> is <code>[#name 'identifier']</code>

Literals

Base building block of every Micro script
Are however almost never explicitly written (but implicitly generated by special syntactic forms)
Syntax : <code>'lit</code> or <code>'`lit with spaces or special chars`</code>

Macros

Are valid expressions
Three styles : block, inline, declarative
Block : <code>block (args) { stmts } limb { stmts }</code> <code>block { stmts } limb { stmts }</code> <code>block (args) stmt limb stmt</code> <code>block stmt limb stmt</code>
Inline : <code>inline (args) limb expr</code> <code>inline arg limb expr</code> <code>inline</code>
Declarative : <code>decl name(args) limb expr { stmts }</code> <code>decl name limb expr { stmts }</code>
where <code>stmts</code> and <code>args</code> are (possibly empty) lists of expressions seperated by semicolons (last one being optionnal)